Improvement Changelog

Predictions were written before each stage ran and are kept visible next to the measurement, including where they were wrong.

Primary metric: **F1 over labelled findings**, matched by (resource address, category) across 15 pull requests carrying 10 real findings and 7 findings a good reviewer suppresses. Scoring is set intersection — no model grades another model anywhere in this project.

Correction, applied after the fact

Every stage below was originally measured once and compared against a single B1′ run scoring **0.900**. That run was later repeated three more times under identical conditions and scored **0.737, 0.762, 0.737** — mean **0.784**, standard deviation **0.078**, range **0.737–0.900**. The number this project compared everything against was the best of four.

Four of the six iterations land *inside* that range and are therefore **indistinguishable from doing nothing**. The mechanism stories originally written for I2, I4 and I5 are not supported by the data and have been struck through rather than deleted. What survives is stated below each one.

This correction is the most useful result in the project. See the hot take.

Follow-up, after the correction: the three stages that fell outside the baseline range were each repeated four times as well. All three survive as real regressions with no overlap against the baseline's own spread. The confirmed distributions are in "Every stage, four runs each" below.

Baseline noise floor: B1′ = 0.784 ± 0.078 (n=4), range 0.737–0.900. A stage is only distinguishable from the baseline if it falls outside that range.

Stage	What ran, and why	Predicted	Measured	Verdict against the noise floor
B0	Raw Checkov, exactly as CI runs it today: whole-stack static scan, no plan, no PR description. The status quo.	F1 0.35	**F1 0.053** — precision 0.028, recall 0.400, **9.5 findings per PR**	Keep as the honest status-quo number
B0′	Checkov scoped to the resources the plan actually changes, as a competent CI integration would. A deliberately *stronger* baseline.	—	**F1 0.320** — precision 0.267, recall 0.400, **0.9 findings per PR**	Keep as the scanner baseline
B1	One direct prompt over the diff and PR description. Sonnet 4.6, no plan, no scanner, no tools.	F1 0.45	**F1 0.783** — precision 0.692, recall 0.900, band C recall 0.86	Keep. Prediction badly wrong; see below
B1′	The same prompt on Haiku 4.5, to test whether the result is about the task or the model.	worse than B1	**F1 0.784 ± 0.078** over 4 runs (0.900, 0.762, 0.737, 0.737)	The bar. The cheap model matched or beat Sonnet
B2	General agent with tools and no task structure: reads the whole working directory, runs the scanner, decides for itself when to stop. Haiku 4.5.	F1 0.55	**F1 0.636** — precision 0.583, recall 0.700, band C recall 0.57, $0.133/PR	Keep. Worse than one prompt, at 19× the cost
I1	Add the plan's changed resources to B1′. Same model, same instructions, same code path — the plan is the only variable.	0.68, the project's central hypothesis	**F1 0.667**, band C recall 0.57	**Remove.** Below every B1′ run (–1.5 sd). Survives the correction
I2	Add the scanner's output to B1′, framed as claims to adjudicate rather than findings to repeat.	regression, via parroting	**F1 0.842**, noise 7 of 7 suppressed	**No measurable effect.** Inside the B1′ range
I3	Drop any finding not tied to the plan's change set or the diff. Deterministic, no model call.	flat	**F1 0.900**, 0 findings dropped, **$0.00**	**No measurable effect**, as predicted. Keep — it is free and cannot reduce recall
I4	Add a computed monthly cost delta to B1′.	improvement on cost	**F1 0.842**, 0 of 2 cost findings	**No measurable effect.** Inside the B1′ range

Stage	What ran, and why	Predicted	Measured	Verdict against the noise floor
I5	Carry review decisions forward between pull requests.	uncertain	**F1 0.800**, noise 7 of 7	**No measurable effect.** Inside the B1′ range
I6	Split the review across security, cost and reliability specialists, merged by a lead.	the worst result in the project	**F1 0.583**, precision 0.500, 1.0 findings per PR	**Remove.** Below every B1′ run (–2.6 sd). The worst configuration measured

Stage notes

B0 — the status quo is worse than predicted, in the way that matters

F1 0.053 against a predicted 0.35. The miss is entirely precision: 0.028. Recall was as expected at 0.400.

The reason is the thing the project exists to fix. Checkov reports on the whole stack on every run, so each of the 15 pull requests draws the same ~9.5 flagged resources regardless of what the change touched. Recall is unaffected, precision collapses, and the practical result is the one every team recognises — a CI check that is red on every pull request and therefore read on none.

Recording this as a failed prediction rather than adjusting the target: the 0.35 estimate was made thinking of Checkov's accuracy on the resources it examines, not of its output volume per pull request. Those are different quantities and only the second one reaches a reviewer.

B0′ — building the strongest fair version of the baseline

Comparing an agent against an unreadable wall of output would be a strawman, so the baseline was strengthened rather than left where it fell. Restricting Checkov to the resources the plan changes is what a diff-aware CI integration does, and it is a much harder thing to beat: findings per pull request fall from 9.5 to 0.9 and F1 rises from 0.053 to 0.348.

B0′ is the number the agent has to beat. B0 is reported alongside it because it is what the user actually experiences today.

B1 — the prediction was wrong by enough to change the project

Predicted F1 0.45. Measured **0.783**, against the scanner's 0.320. The pre-registered target for the *finished agent* was 0.80, and a single prompt over the diff essentially reaches it.

The number that matters is band C. Those six cases exist because a static scanner is structurally blind to them — cross-file effects, plan transitions, cost, a removed guardrail — and Checkov scores 0.14 there. A one-shot prompt scores **0.86**. Reading the findings shows why: the model knew from the diff alone that `storage_encrypted` is immutable on `aws_db_instance` and forces replacement, that `desired_capacity = 1` under `min_size = 3` will be reverted by the autoscaling service, that a second NAT gateway costs real money, and that removing `prevent_destroy` matters even though the plan shows nothing.

This substantially disproves the I1 hypothesis. The premise of iteration I1 was that an agent needs `plan.json` to see what the diff hides. It turns out a capable model infers most of those transitions without it. Structurally blind to a *scanner* is not the same property as hard for a *model*, and the case set was designed around the first while the metric measures the second.

What remains is more interesting than what was lost. The headroom is no longer recall (0.900 — the baseline finds nearly everything) but **precision (0.692) and verdict accuracy (0.733)**. B1 reprints 3 of 7 findings a good reviewer suppresses: it flags plain HTTP on a listener that never leaves the VPC, and a staging bucket whose contents expire in a day. The remaining work is judgment and restraint, not detection.

The project's question therefore changes from *can an agent beat a linter* — answered, trivially, by one prompt — to **does agent machinery add anything over a competent one-shot reviewer?** That is a harder question and a more honest one, and "mostly no" is a publishable answer.

Consequences for the remaining stages, decided now rather than after seeing which way the numbers fall:

- I1 (plan JSON) is now expected to add little. It runs anyway, because a measured null result on the project's own central hypothesis is worth more than a quietly dropped stage.
- I2 (linter output as evidence to adjudicate) and I3 (citation verification) target precision, which is where the headroom actually is. They are promoted ahead of I1 in importance.
- The pre-registered target of F1 0.80 is retained rather than raised. Moving a target after seeing the baseline is how a project talks itself into a result.

B1′ — the cheaper model beat the more expensive one, and the reason is restraint

Run to answer a narrow question: is B1's 0.783 a fact about the task or about Sonnet 4.6? The prediction was that Haiku 4.5, at a third of the price, would do noticeably worse on the judgment-heavy cases.

It did better. **F1 0.900 against 0.783**, at $0.11 for the full fifteen cases versus $0.22, and faster per case.

Where the two models are identical:

	Sonnet 4.6	Haiku 4.5
recall	0.900	0.900
band C recall	0.86	0.86
band A recall	1.00	1.00
false blocks on clean PRs	0	0

They find the same things. Detection is not what separates them.

Where they differ, in opposite directions:

	Sonnet 4.6	Haiku 4.5
precision	0.692	**0.900**
noise correctly suppressed	4 of 7	**7 of 7**
verdict accuracy	**0.733**	0.667
findings per PR	0.9	0.7

Sonnet reprints three findings a good reviewer suppresses — plain HTTP on a listener that never leaves the VPC, an internal load balancer sharing a security group, a staging bucket whose contents expire in a day. Haiku suppresses all seven and says nothing about any of them.

Haiku's failure is the mirror image. On cases 01, 02, 03, 09 and 10 it finds the right problem and then files it as `warn` where the label says `block`. It calls an unrestricted `Action: "*"` administrative policy a warning. It finds the right thing and under-calls it.

So neither model is better at reviewing. One over-reports and escalates correctly; the other under-reports nothing and under-escalates everything. Both failure modes are calibration, and both look addressable by prompt and verification rather than by model capability — which is the strongest evidence so far for where the remaining work actually is.

B1′ becomes the bar the agent has to beat, on the same principle that made scoped Checkov the baseline rather than raw Checkov: compare against the strongest fair version of the alternative. That means the agent now has to beat F1 0.900 at $0.007 per pull request, which is a far harder target than the 0.320 it started with and than the 0.80 that was pre-registered. The pre-registered target is left where it is; the bar is what moved.

Every stage, four runs each

After the correction, every stage that had appeared to regress was repeated until it had the same number of samples as the baseline.

stage	four runs	mean	sd	range	vs baseline
B1′ baseline	0.900, 0.737, 0.762, 0.737	**0.784**	0.078	0.737–0.900	—
I1 plan	0.667, 0.667, 0.636, 0.727	0.674	0.038	0.636–0.727	–**0.110, no overlap** (t = –2.5)
I6 multi-agent	0.583, 0.667, 0.636, 0.727	0.653	0.060	0.583–0.727	–**0.130, no overlap** (t = –2.6)
B2 agent + tools	0.636, 0.545, 0.560, 0.609	0.588	0.042	0.545–0.636	–**0.196, no overlap** (t = –4.4)

All three hold. No stage's range touches the baseline's, and B2 — the most capable configuration built, with tools, the plan, the scanner and unlimited steps — is the furthest from it.

One detail worth reading off the numbers: the *stages* are less variable than the baseline (sd 0.038–0.060 against 0.078). Three of the four baseline runs cluster tightly at 0.737, 0.737 and 0.762; the 0.900 that this project reported for most of its life is an outlier in its own sample. Drop it and the baseline is 0.745 ± 0.014 — which makes the regressions cleaner still, and the original single-run report worse.

The four stages inside the baseline range (I2, I3, I4, I5) were left at one run each. Distinguishing a 0.05 effect from this noise floor would need roughly an order of magnitude more samples than the difference is worth, and reporting them as "no measurable effect at n=1, against a baseline of 0.784 ± 0.078" is the honest description of what was actually established.

The variance check, and what it cost this project

Three additional B1′ runs, identical in every respect to the first:

run	F1	cost findings found
1 (the one reported throughout)	**0.900**	1 of 2
2	0.737	1 of 2
3	0.762	1 of 2
4	0.737	**0 of 2**

Mean 0.784, standard deviation 0.078. **The baseline this project measured everything against was the best of four samples**, and no repeat measurement was taken until every stage had already been run, written up and committed.

Placing each stage against that distribution:

stage	F1	verdict
scoped Checkov	0.320	below every run (–5.9 sd) — **real**
I6 multi-agent	0.583	below every run (–2.6 sd) — **real**
B2 agent + tools	0.636	below every run (–1.9 sd) — **real**
I1 plan	0.667	below every run (–1.5 sd) — **real**
I5 memory	0.800	inside the range — **no effect**
I2 scanner	0.842	inside the range — **no effect**
I4 cost	0.842	inside the range — **no effect**
I3 verification	0.900	inside the range — **no effect**

Four of six iterations cannot be distinguished from doing nothing. Three regressions survive, and they are the three largest.

The I4 cost finding does not survive, and it was the headline. The claim was that supplying a cost figure drove cost findings from 1 of 2 to 0 of 2. Run 4 of the *unmodified baseline* also found 0 of 2. With two labelled findings in that category, "1" and "0" are the same measurement. The observation that the model wrote "expected cost" while approving a $438/year increase is still real and still worth reading — but it is an anecdote about one response, not evidence that the cost table caused anything.

What survives from the earlier analysis is narrower and better supported: the case 08 recategorisation from `data-loss` to `reliability` happened in both I1 and B2, which reached the plan by different routes, and both of those stages are independently below the noise floor.

I4 — the intervention aimed at the known weakness made the weakness total

Cost was the one category every configuration kept dropping, and the diagnosis was attention rather than ability: B1′ can price a NAT gateway, it just does not always think to. So I4 supplies the number directly — a computed monthly and annual delta for the resources the plan creates and destroys, from `tools/pricing.py`. It was the only stage in this project with a mechanism arguing in advance that it should help.

It found zero cost findings. B1′, given no cost information whatsoever, found one.

Case 11 is the whole result in one row. The pull request adds a second NAT gateway, about $438 a year, and its description mentions only availability.

	cost data supplied	verdict	headline
B1′	none	**warn**	"Good reliability improvement, but adding NAT gateway doubles nat…"
I2	none (scanner output instead)	approve	"…no problems with the implementation"
I4	**+$438/year, stated explicitly**	approve	"High-availability NAT improves reliability with **expected cost**"

Shown the number, the model called the cost *expected* and approved. Not missed — acknowledged, and dismissed.

The mechanism is worth stating carefully, because it is the most transferable thing this project found. **Information that answers a question also removes the reason to ask it.** A reviewer flags an unannounced cost because nobody has accounted for it; the finding is the surprise, not the arithmetic. Presenting the figure as a computed, labelled section of the prompt made it read as already accounted for. The harness did the noticing, so the reviewer stopped.

Two honest limits on this claim. The cost category carries only two labelled findings, so the effect is legible rather than statistically strong — it is case 11's headline, in the model's own words, that makes it more than noise. And the table genuinely could not reach case 09, where a NAT gateway is orphaned rather than created and so has no cost delta; that limitation was written into `pricing.py` before the run rather than discovered in the results.

Decision: remove. Four additions, four regressions.

I2 — the prediction was right, the mechanism was wrong, and the result is sharper for it

Recorded before this ran: *"I2 will also regress. It adds context of exactly the kind that has now failed twice, and the failure mode it invites — defending the scanner's findings rather than judging them — is the one flagged in the original plan."*

It regressed: **F1 0.842 against B1′'s 0.900.** But the predicted mechanism did not happen at all. The "claims to adjudicate" framing worked exactly as intended — **noise suppression stayed at 7 of 7** and precision held at 0.889 against 0.900. The agent did not parrot the scanner once.

It lost on recall instead: 0.900 → 0.800. Breaking the loss down by category makes it unambiguous.

category	labelled	B1′ found	I2 found	scanner has a check?
network-exposure	3	3	3	yes
privilege-escalation	2	2	2	yes
data-loss	1	1	1	no
guardrail	1	1	1	no
reliability	1	1	1	no
cost	**2**	**1**	**0**	**no**

Every category held at parity except one. **The entire regression is the cost findings, and I2 lost all of them.** On case 11 — a pull request that adds a second NAT gateway, roughly $400/year, described only as an availability improvement — B1′ raised it as a cost finding and I2 returned `approve` with nothing to say. The scanner had reported nothing against that pull request's resources, because Checkov has no concept of money.

The mechanism is not parroting but **anchoring**. Shown the output of a security scanner, the review became security-shaped. Risk-shaped categories the scanner also cannot see — data loss, a removed guardrail, a fleet that will not scale — all survived, because they still look like the kind of thing a scanner is for. Cost is the only labelled category that is not a risk at all, and it is the one that fell out.

Decision: remove. Not because the framing failed — it succeeded, and that is worth keeping as a finding in its own right: telling a model it may rule against a tool does stop it deferring to that tool. But supplying the tool's output at all narrowed what the model went looking for, and the cost was higher than the benefit, which was zero because B1′ already suppressed perfectly.

I1 — the plan makes the review worse, and B2 had already shown it

This is the project's central hypothesis: a reviewer needs the plan, because the diff does not say what will happen. I1 tests it as cleanly as the harness allows — the plan is added as a flag on the *same* runner, so the model, the review instructions, the output schema and the code path are byte-identical to B1′ and the plan is the only variable.

F1 fell from 0.900 to 0.667.

	B1′ no plan	I1 with plan	B2 with plan + tools
F1	**0.900**	0.667	0.636
precision	**0.900**	0.636	0.583
recall	**0.900**	0.700	0.700
band C recall	**0.86**	**0.57**	**0.57**
verdict accuracy	**0.667**	0.467	0.600
noise suppressed	**7 of 7**	5 of 7	6 of 7

Band C is where the plan was supposed to help — those six cases were built because a scanner cannot see across files or into plan transitions. Recall there drops from 0.86 to 0.57, and it drops to **exactly** 0.57 in both I1 and B2, which reached the plan by completely different routes: I1 was handed the changed resources in its prompt, B2 chose to open `plan.txt` with a tool. Two architectures, one effect, the same number.

The mechanism is visible on case 08, the case built specifically to require the plan:

	verdict	category	
B1′ (no plan)	block	`data-loss`	correct
I1 (plan in prompt)	block	`reliability`	wrong
B2 (plan read by tool)	warn	`reliability`	wrong

Working from the diff alone — `storage_encrypted` flipping from false to true — the model reasons about what happens to the data and calls it data loss. Shown the plan, which states the action as `delete` then `create`, it reasons about the *operation* and calls it reliability. The plan is more precise and more truthful than the diff, and it reframes the question from "what happens to the data" to "what happens to the resource". Both models that saw the plan made the same substitution.

That is a real mechanism, not a scoring artifact: the finding is on the right resource with the right verdict, and only the category moved.

Decision: remove. I1 is the stage the project was built around, and it is being cut on its own evidence. The prediction going in was already pessimistic after B1 — recorded as "expected to add little" — but the measured result is worse than null, and it replicates.

B2 — more capability made the review worse

B2 gets strictly more than B1: the whole working directory (Terraform sources, the human-readable `plan.txt`, the raw plan JSON, the prior state), the scanner on demand, and as many steps as it wants. It used 7.2 tool calls per pull request on average.

It scored **F1 0.636**, against 0.900 for the same model given one prompt and nothing but the diff. It cost **$0.133 per pull request against $0.007 — 19× more for a materially worse review.**

	B1′ one prompt	B2 agent with tools
F1	**0.900**	0.636
precision	**0.900**	0.583
recall	**0.900**	0.700
band C recall	**0.86**	0.57
verdict accuracy	**0.667**	0.600
cost per PR	**$0.007**	$0.133
steps per PR	1	7.2

Two failure modes, both visible in the trajectories.

It wandered. On case 05 — a pull request that adds an internal load balancer — it reported a finding against `aws_db_instance.main`, which that pull request does not touch. On case 07, a staging bucket, it reported against `aws_iam_role_policy.app_data`. Both are resources it found by reading files it was free to read. The review instructions say in as many words that pre-existing conditions elsewhere in the stack are not findings; having the whole directory available made that instruction harder to follow, not easier. Two of its thirteen findings are on resources absent from the plan's change set. B1 has none.

Reading the plan did not help it classify. Case 08 is the case built to require the plan, and `plan.txt` states `aws_db_instance.main must be replaced` in its first lines. B2 read that file, named the right resource, and filed it as `reliability` rather than `data-loss`. It saw the replacement and did not recognise it as data loss. B1, working from the diff alone with no plan at all, categorised it correctly.

Band C recall falling from 0.86 to 0.57 while gaining access to the plan is the single most direct evidence in this project against the I1 hypothesis. The premise was that the agent needs the plan. B2 had the plan, read the plan, and did worse.

The tentative reading, to be tested by the agent stages rather than asserted here: tool access broadens what the model attends to, and this task rewards narrowness. A reviewer's job is to answer "what does *this change* do", and every additional file in context is an invitation to answer a different question.

A case in the set contained a real finding, and B1 found it

Case 05 tests whether the agent suppresses scanner noise about an internal load balancer. The original overlay added `aws_lb.internal` with no listener and no target group — which is a genuine reliability defect, since a load balancer with no listener routes nothing. B1 reported it and was scored as a false positive for being correct.

A case meant to measure suppression cannot contain a real problem. The overlay now includes the target group and listener that make the change complete, and the case was re-planned, re-scanned and re-run. Scoped Checkov moved 0.348 → 0.320 and B1 0.818 → 0.783 as a result; both are reported at the corrected values throughout.

This is the second defect found by reading outputs rather than scores, and the second correction that made the agent's job harder rather than easier.

The scoring methodology was wrong three times

Recorded because all three were caught by looking at outputs rather than scores, and every fix raised a baseline rather than the agent.

1. **Address-only matching gave the scanner free recall.** Scoring the baseline generously — credit for naming the right resource, however it classified the problem — put Band C recall at 0.43. Inspecting it showed why: Checkov flags `aws_s3_bucket.data` on every single case for cross-region replication, event notifications and KMS encryption, and case 15's finding happens to sit on that same resource. It was being credited for naming the right resource for entirely unrelated reasons. Replaced with a published check-id → category map (`tools/run_checkov.py`), under which Band C recall falls to 0.14.

2. **An input variable has three legitimate names and only one was accepted.** Sonnet wrote `variable.app_data_bucket_arns`, Haiku wrote `var.app_data_bucket_arns`, and the label carried the bare name — so case 12 scored zero for both while both had found exactly the right thing. Which form a reviewer picks says nothing about review quality, so `norm()` now strips the `var.` and `variable.` prefixes. This raised Haiku from 0.800 to 0.900 and had already raised Sonnet from 0.727 to 0.818.

3. **Noise labels were matched by address *and* category, so miscategorising a noise finding laundered it.** Checkov reports the case 06 public-bucket resources as `network-exposure` while the label carries category `noise`, so the keys never met and the harness reported 6/6 noise correctly suppressed when the true figure was 1/6. Noise now matches on address alone: raising that resource at all is the error, whatever it is filed under.

One prediction about the case set was wrong

Case 12 widens an IAM policy by editing a variable default three files away; the policy resource never appears in the diff. It was labelled `linter_catches: false` on the assumption that a static scanner would not resolve the variable.

Checkov resolves it and fires `CKV_AWS_355` on `aws_iam_role_policy.app_data`. It is the only Band C case the scanner finds, and it is the whole of Checkov's 0.14 Band C recall. The label has been corrected to `linter_catches: true` and the case kept in Band C, since what it tests — reasoning about an effect absent from the diff — is still what separates it from Band A for the diff-reading baselines.

Every stage measured so far

	Scoped Checkov	B1 Sonnet	B1′ Haiku	I1 plan	I2 scanner	I4 cost	B2 tools
F1	0.320	0.783	**0.900**	0.667	0.842	0.842	0.636
precision	0.267	0.692	**0.900**	0.636	0.889	0.889	0.583
recall	0.400	0.900	**0.900**	0.700	0.800	0.800	0.700
verdict accuracy	0.467	**0.733**	0.667	0.467	0.600	0.533	0.600
band C recall	0.14	0.86	**0.86**	0.57	0.71	0.71	0.57
noise suppressed	1 of 7	4 of 7	**7 of 7**	5 of 7	**7 of 7**	**7 of 7**	6 of 7

	Scoped Checkov	B1 Sonnet	B1′ Haiku	I1 plan	I2 scanner	I4 cost	B2 tools
cost findings	0 of 2	1 of 2	**1 of 2**	0 of 2	0 of 2	**0 of 2**	0 of 2
false blocks	1	0	0	0	0	0	0
cost per PR	$0	$0.015	**$0.007**	$0.009	$0.007	$0.007	$0.133
steps per PR	—	1	1	1	1	1	7.2

Nothing beat the simplest configuration. The best reviewer measured in this project is one prompt, the diff, the pull request description, and the cheapest model available, at $0.007 per pull request — and three of the six additions built on top of it made it measurably worse.

Every addition after the simplest possible thing has made the result worse. The best reviewer measured in this project is one prompt, the diff, the pull request description, and the cheapest available model.

Band C is the project's reason for existing. The scanner scores 0.14 there; both models score 0.86 from the diff alone. The gap between the scanner and a single prompt is enormous; the gap between a single prompt and anything more elaborate is what remains to be shown.

Model access: what actually runs, and why

Runs go through **Amazon Bedrock** on AWS account 313951301623, authenticated by the ordinary AWS credential chain, so no API key exists anywhere in this repository. Reproducing this needs an AWS account with Anthropic model access, not an Anthropic API key.

Three things cost time here and are recorded so nobody repeats them:

1. **Bedrock serves newer Claude models through inference profiles.** The id must carry a `us.` prefix and go through the `bedrock-runtime` InvokeModel path (`AnthropicBedrock`), not the Messages/Mantle endpoint (`AnthropicBedrockMantle`). Measured on this account:

id	result
`anthropic.claude-sonnet-5`	403 on Mantle
`us.anthropic.claude-sonnet-5`	404 on Mantle, AccessDenied on InvokeModel
`us.anthropic.claude-sonnet-4-6`	works

2. `ListFoundationModels` **returns ids for the wrong endpoint.** It lists dated forms like `anthropic.claude-haiku-4-5-20251001-v1`, which 404 on the Messages endpoint. Copying ids from that listing makes things worse.

3. **The Claude 5 family is not available on either account tested.** Sonnet 5, Opus 5, Fable 5, Opus 4.7 and Opus 4.8 all return AccessDenied; 4.6 and 4.5 are granted. The Bedrock console's "Model access" page has been retired and its replacement auto-enables on first invocation, so there is no form to fill in -- the denial is account eligibility, not a missing opt-in.

So the reported runs use **Claude Sonnet 4.6** (`us.anthropic.claude-sonnet-4-6`, $3/$15 per MTok) and the cheap comparison runs use **Claude Haiku 4.5** (`us.anthropic.claude-haiku-4-5-20251001-v1:0`, $1/$5). Opus 4.6 is available and deliberately unused: two models answer "does the cheaper one hold up", and a third only adds spend.

Verified on this path: adaptive thinking, `output_config.effort`, and structured outputs all work on Sonnet 4.6. Haiku 4.5 rejects `effort` and takes `thinking: {type: "enabled", budget_tokens: N}` instead, which `tools/model.py` handles.

A deviation from the plan, recorded

B2 was specified as "a general agent with shell access". It was built with a confined tool surface instead: it can list and read files inside one case directory and run the scanner with fixed arguments, but no command it writes is executed. The runs happen on a workstation holding live cloud credentials and a git remote with push rights, and handing a model an unrestricted shell there was not a decision to take quietly.

The substitution is judged not to weaken the baseline: B2 still receives strictly more information than B1 and still chooses its own path through it. A shell would have added the ability to run arbitrary commands over the same files it can already read. If that judgement is wrong, the fix is to rerun B2 inside a container, and the result above should be read as a lower bound on what an unconstrained agent would do.

Hot take, provisional

A single run is not a measurement, and this project nearly published four that were not.

Six iterations were built, run once each, and compared against one baseline number. Four regression stories were written, each with a plausible mechanism, each committed with confidence. Then the baseline was repeated three times and scored 0.737, 0.762 and 0.737 against the original 0.900 — and three of those four stories evaporated, including the one with the best narrative and the most quotable model output.

Nothing about the method was careless in an obvious way: the scoring is deterministic, the labels were fixed before any run, no model grades another model, and every prediction was recorded in advance. The missing control was the cheapest one available — running the same thing twice. It cost $0.33 and twelve minutes, and it should have been the second measurement in the project rather than the twenty-second.

The reason it is easy to skip is that a temperature-zero-feeling pipeline *looks* deterministic. Structured outputs, a fixed prompt, set-intersection scoring: everything downstream of the model is reproducible, which quietly suggests the model is too. On 15 cases carrying 10 findings, one finding moving is 0.05–0.10 of F1. The noise floor was wider than five of the eight effects being measured.

What we would do differently: measure the noise floor before measuring anything else, and refuse to interpret any difference smaller than it.

The secondary finding, which does survive:

Context is not free, and it is not neutral. Adding it reframes the question.

Three additions, three regressions, three different reframings — and in each one the model started answering a slightly different question than the one asked:

addition	what the review became about	what fell out
the terraform plan	the resource lifecycle	data loss (case 08 recategorised)
a security scanner's output	security	every cost finding
a computed cost table	**money already accounted for**	**every cost finding**
file and scanner tools	the codebase	focus on the change itself

Of these, only the plan row and the tools row survive the variance check as measured effects. The scanner and cost rows are inside the noise floor and are listed as observations about individual responses, not as measured regressions. The cost table's "expected cost" approval remains the single most striking thing a model said in this project, and it remains unproven.

The plan is strictly more accurate than the diff. It is machine-generated, it is what Terraform will actually do, and it states the RDS replacement in as many words. Supplying it made the review worse in two independent architectures, by the same amount, through the same substitution: shown a `delete` followed by a `create`, the model reasons about the resource lifecycle and stops reasoning about the data inside it.

The lesson generalises past Terraform. When an agent underperforms, the reflex is to give it more — more context, more tools, more steps. Every such addition also changes what the model thinks it is being asked. Three times in this project the addition moved attention somewhere defensible and somewhere wrong: the plan moved it from data to resources, tools moved it from the change to the codebase, and the only configuration that stayed on the question was the one with the least information.

What we would build differently next time: measure the naive version first and treat every subsequent addition as a hypothesis that must beat it, rather than as progress. This project pre-registered six iterations on the assumption they would stack. Two have been measured and both regress.

What one thing did work

I2's framing. Told that the scanner's output was a set of claims it was expected to rule against where context warranted, the agent suppressed 7 of 7 and deferred to the scanner zero times. The original plan predicted the opposite -- that naming a finding would make a model defend it. That prediction was wrong, and the technique is worth keeping even though the stage that carried it is not.

Still to run

I3 (citation verification), I5 (review memory), I6 (the multi-agent split).

Predictions, recorded before running. I3 only *removes* findings that cannot cite real evidence, so it cannot lose recall it does not already have; B1′ has no invented citations to remove, so the expectation is a flat result, which is the point -- a technique that does nothing on a clean baseline is still worth measuring once. I6 adds orchestration on top of a task where every addition so far has hurt, and is expected to be the worst result in the project.

The measured headroom is verdict accuracy, which no configuration has taken above 0.733, and where nothing tried so far has been aimed.